

Zod Validation Architecture

Single contract, three consumers — schemas in `packages/shared`, validated at every trust boundary, producing both runtime checks and static types.

1 Why Zod, Why Centralized

Decision drivers (parallel single-seam pattern documented in ADR-002-ai-provider.md):

- **One source of truth.** A field added to `Task.input` lands on the API validator, the FE form, and the typed API client from a single edit.
- **Runtime + types.** `z.infer<typeof schema>` gives the TS type; `schema.parse(...)` gives the runtime check. Same definition, no codegen.
- **Composability.** `.extend` / `.omit` / `.partial` / `.merge` handle schema evolution without rewriting consumers.
- **Cheap error shape.** `.flatten()` maps cleanly onto the 400 `VALIDATION_ERROR` envelope.

Alternatives rejected:

io-ts	Extra runtime, weaker TS inference, no RHF resolver
Hand-written interfaces	Duplication drift between FE form, FE client, BE validator
JSON Schema + codegen	Heavier toolchain, weaker composition, two-file edit per change

2 Package Layout

```
packages/shared/src/
common.ts      # primitives: email, password, slug, cuid, pagination, string bounds
pagination.ts  # CursorPaginationQuery + CursorPaginationEnvelope + encode/decode
user.ts        # registerSchema, loginSchema, userPublicSchema, changePasswordSchema
auth.ts        # authResponseSchema, refreshTokenSchema, oauthCallbackSchema
workspace.ts   # workspace CRUD + member schemas
task.ts        # createTaskSchema, updateTaskSchema, moveTaskSchema, taskSchema, ...
comment.ts     # createCommentSchema, commentSchema, listCommentsQuerySchema
label.ts       # label CRUD schemas
notification.ts # notification + list query schemas
attachment.ts  # attachment response schemas
index.ts       # barrel re-export (no barrel imports inside the package)
```

Build target (`tsup.config.ts`): dual ESM/CJS with per-module subpath exports. Subpath imports keep individual module graphs small.

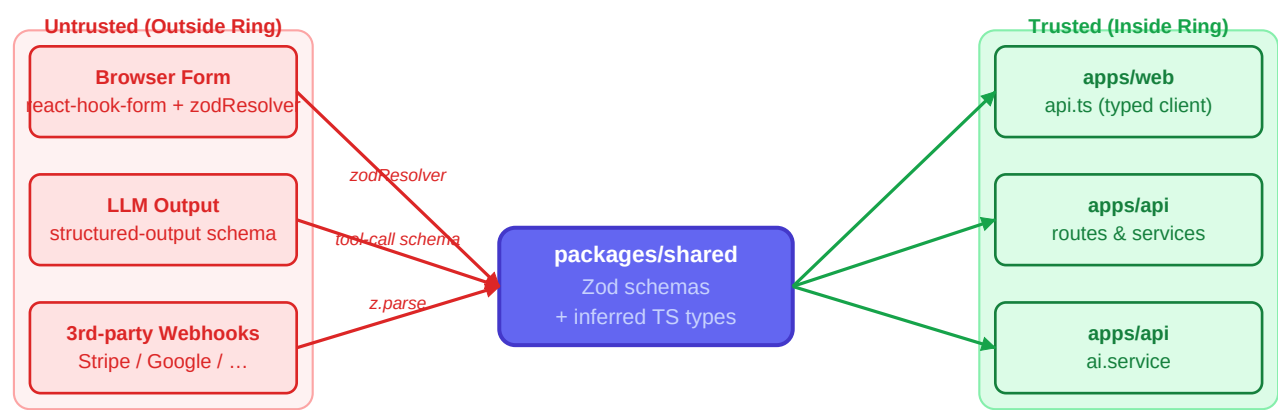
Naming Conventions

Primitive	common.ts — email, password, slug, cuid, pagination
Entity	bare <code><entity>Schema</code> — wire shape only

Input	<verb><Entity>Schema — createTaskSchema, updateTaskSchema
Query	list<Entity>QuerySchema or extends CursorPaginationQuery
Type	z.infer alias exported beside every schema

3 The Two-Ring Contract

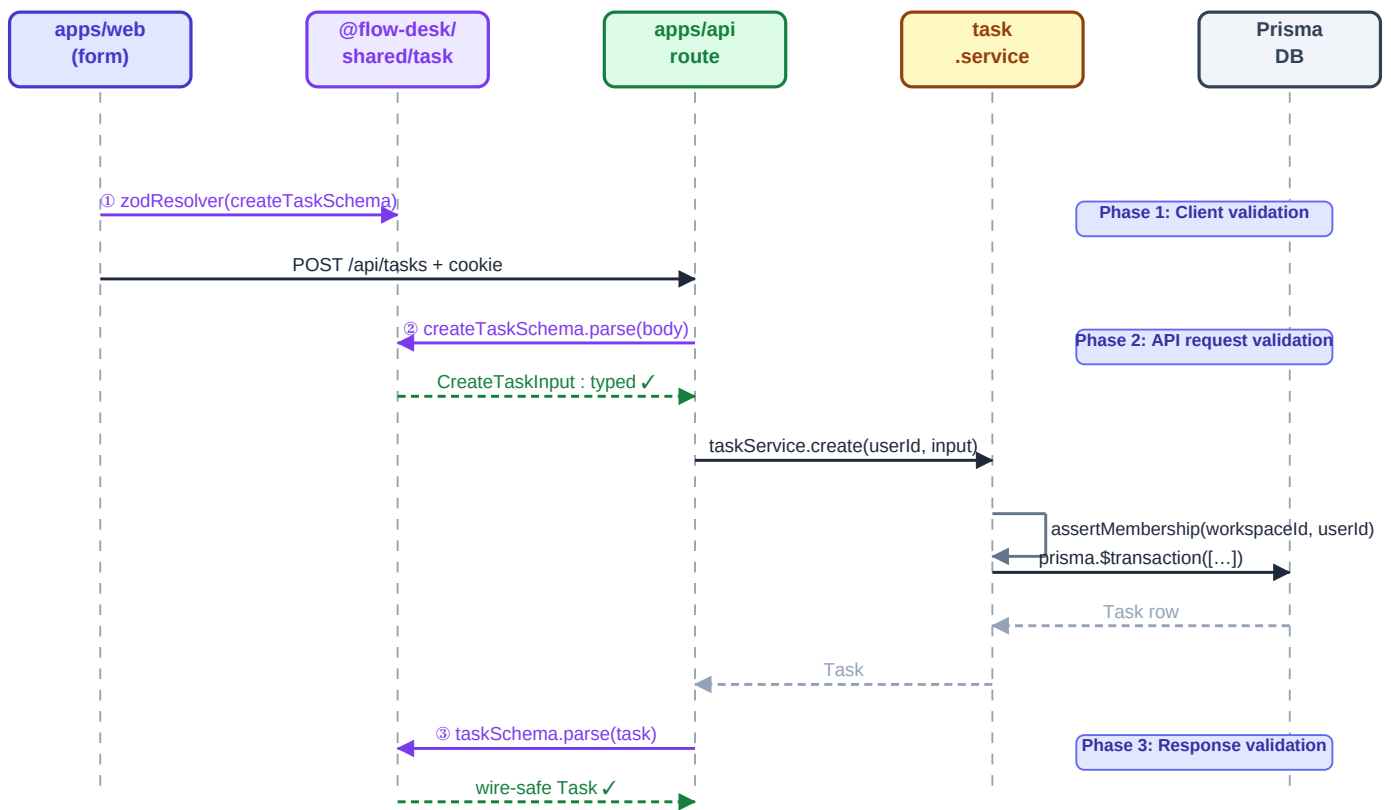
Two rings, one definition. Everything outside the process is **untrusted** and parsed through a Zod schema before any business logic runs. Everything inside is **trusted** — inputs have already been parsed; the service uses the inferred TS type and never re-parses.



4 Request Lifecycle — Create Task

Validation happens **three times**, on purpose:

Phase	Where	Tool	Purpose
1 — Client form	apps/web	zodResolver(formSchema)	Instant UX feedback
2 — API request	apps/api route	zValidator / .parse(body)	Authoritative. No bypass.
3 — API response	apps/api route	taskSchema.parse(task)	Catches Prisma ↔ wire drift



Forms too simple for react-hook-form rely on API request-time validation only. The cost of skipping client-side parse is one extra round-trip; the cost of duplicating schemas is drift.

5 Error Pipeline

Every Zod failure hits the same path. Two intentional error shapes exist:

Code	HTTP	Source	Meaning
INVALID_QUERY	400	@hono/zod-validator middleware	Query-string / param parse fail (programmer bug)
VALIDATION_ERROR	400	errorHandler catches ZodError	Body / param fail (user input bug)

The split exists because query-string failures usually indicate a **programmer bug** (broken link / bookmark), while body failures usually indicate a **user input bug**.

6 Canonical Patterns

Extend a primitive

```
// packages/shared/src/workspace.ts
import { paginationSchema, cuidSchema } from './common';

export const listWorkspacesQuerySchema = paginationSchema.extend({
  ownerId: cuidSchema.optional(),
});
```

Evolve without breaking

```
// Before: title was required.
export const updateTaskSchemaV1 = z.object({ title: nameSchema });

// After: all fields partial. Existing callers still typecheck.
export const updateTaskSchemaV2 = z.object({ title: nameSchema.optional() });
```

When a field's type changes (not optional), bump the versioned handler and ship a migration in `feature_list.json`. Avoid silent coercion unless the source is untrusted-by-design.

Coerce at boundaries only

Query strings	<code>z.coerce.number().int().min(1)</code>	Query is always text
Body	Never <code>.coerce</code> for user-controlled input	FE sends typed JSON
Dates	<code>z.string().datetime({ offset: true })</code>	ISO 8601 on wire; Date only in service

Optional vs Nullable

- `.optional()` — field may be absent (PATCH body fields).
- `.nullable()` — field is present, value is null (e.g. `assigneeId: cuidSchema.nullable()` clears the assignee).
- `.nullish()` — both. Rarely needed; default to one or the other.

Discriminated unions for variant entities

```
export const notificationSchema = z.discriminatedUnion('kind', [
  z.object({ kind: z.literal('TASK_MENTIONED'), taskId: cuidSchema, mentionerId: cuidSchema }),
  z.object({ kind: z.literal('TASK_DUE_SOON'), taskId: cuidSchema, dueDate: z.string() }),
]);
```

Use `discriminatedUnion` whenever the shape genuinely branches. union is fine for memory-less variants but loses the narrowing.

7 Repo + Service Seams

Repositories accept schema-inferred input — they never see `z.ZodType<unknown>`. They return Prisma rows or narrowed projections.

Services orchestrate. They may `.parse()` a third-party response (LLM JSON, outbound webhook) before using it.

Routes are the only layer where raw HTTP shapes touch the codebase. Past the route, everything is typed.

8 Env + Config

- All env vars get a default or `.optional()` — no `process.env.X ?? ''` scattered in route code.
- Coerce numerics (PORT, LLM_MAX_TOKENS); strings for booleans transformed at parse time.

apps/web/src/lib/env.ts is a typed wrapper over `import.meta.env` and deliberately does not use Zod — Vite already validates at build time.

9 LLM Tool / Structured Output

llm-provider.ts calls the upstream with a `response_format` schema borrowed from `@flow-desk/shared` (no inline Zod in the AI module). The returned JSON is re-parsed through the same schema; if the parser throws, it becomes 502 LLM_UPSTREAM. Drift is impossible — the schema is one object.

10 Testing

- **Happy path.** `schema.parse({...valid})` returns the value with the right shape.
- **Each constraint.** One assertion per `.min/.max/.regex/.refine` clause.
- **Boundary values.** Off-by-one on `min(1)` vs `min(0)`, empty string vs `whitespace-after-trim`.
- **Round-trip.** For input schemas, parse and re-stringify; assert no field is dropped.

Route-level tests call the real router and assert error envelope shape on bad input. They do not redo schema tests.

11 What Goes Wrong Without This

- **Schema drift.** FE sends `{ dueDate: 'tomorrow' }`, BE rejects ISO. Fix lives in either place; the other breaks.
 - **Two validation paths.** Client validates, BE trusts. Always validate at BE — client validation is UX, not security.
 - **Validation in services.** Routes are the right answer — they're the trust boundary.
 - **any to escape.** Write a transformation schema or a select projection instead.
-

12 Conventions (Short List)

- Schemas in `@flow-desk/shared/<domain>`. No schemas in `apps/api/src/modules/<feature>/`.
 - Subpath imports: from `'@flow-desk/shared/task'`. Never from inside the package.
 - One file per schema domain (`task.ts`, not `task.schemas.ts`).
 - Both the schema and the inferred type are exported. Types are the consumer API.
 - `flatten()` for human error envelopes; `format()` only for logs.
 - Validation at the edge. Past the route, the types are trusted.
 - Tests in `packages/shared/src/<file>.test.ts` run with the package's vitest config.
 - Constants next to the schema they constrain (`TASK_TITLE_MAX = 100` in `task.ts`).
-

13 Cross-References

docs/ARCHITECTURE.md	Module anatomy and middleware order
packages/shared/src/	The contracts
apps/api/src/shared/middleware/error-handler.ts:33	ZodError mapping
apps/api/src/modules/task/task.routes.ts	Canonical request + response validation pattern
apps/api/src/shared/lib/env.ts	Env-time Zod usage
ADR-002-ai-provider.md	Single-seam LLM provider that consumes shared schemas